

DS 3100

Fundamentals of Data Science

Rmarkdown and R Basics

"IF OPPORTUNITY DOESN'T KNOCK, BUILD A DOOR."

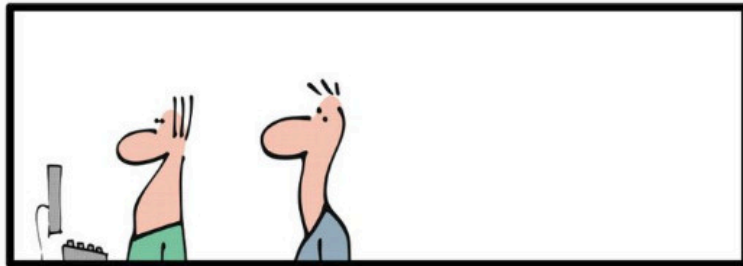
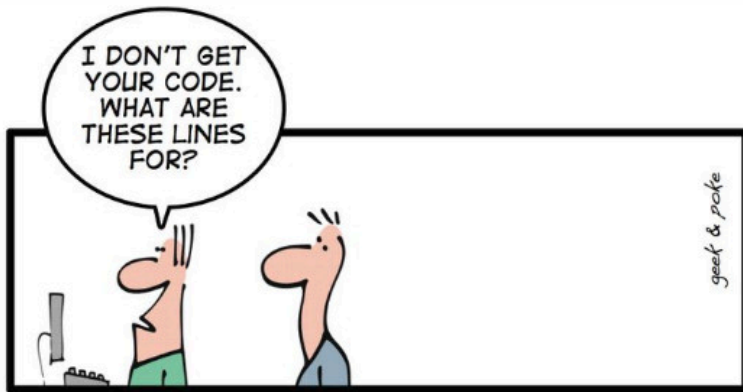
– MILTON BERLE

Rmarkdown

- **Organized code**
- **Professional reports**
- **Seamless integration of analyses**



Part 1



R Basics

Loading data

Wrangling data

[D] What's least favorite part of your job as an MLE/Data engineer/ Data scientist?

Discussion

When I dreamed of becoming a machine learning engineer, I didn't envision that monitoring and debugging performance regressions, backfilling data, or performing migrations/model would fill a significant proportion of my time.

What's the one bit of your job that makes it a little harder to get out of bed in the morning?



Erosis • 5d ago

Dealing with poorly/non-existent labeled data that is too obscure to pseudo label with existing models and you get to the point where you just end up having to hand label everything...

+ 86 ↓ Reply Award Share ...



WhyDoTheyAlwaysWin • 5d ago • Edited 5d ago

Reading / Refactoring / Debugging / Deploying a jupyter notebook made by some DS with a PhD in Physics but no SWE experience.

The solutions they come up with are too complex and convoluted. The stakeholders can't understand it. The available data can't support it. The MLEs and DEs can neither sustain, maintain nor scale it.

I swear this field needs less PhD holders and more statisticians & software engineers.

+ 28 ↓ Reply Award Share ...



giuliano0 • 5d ago

Dealing with corporate that doesn't understand I can't open the effing model file, edit a weight here and there and get rid of bad results just like an if statement would in an app or service.

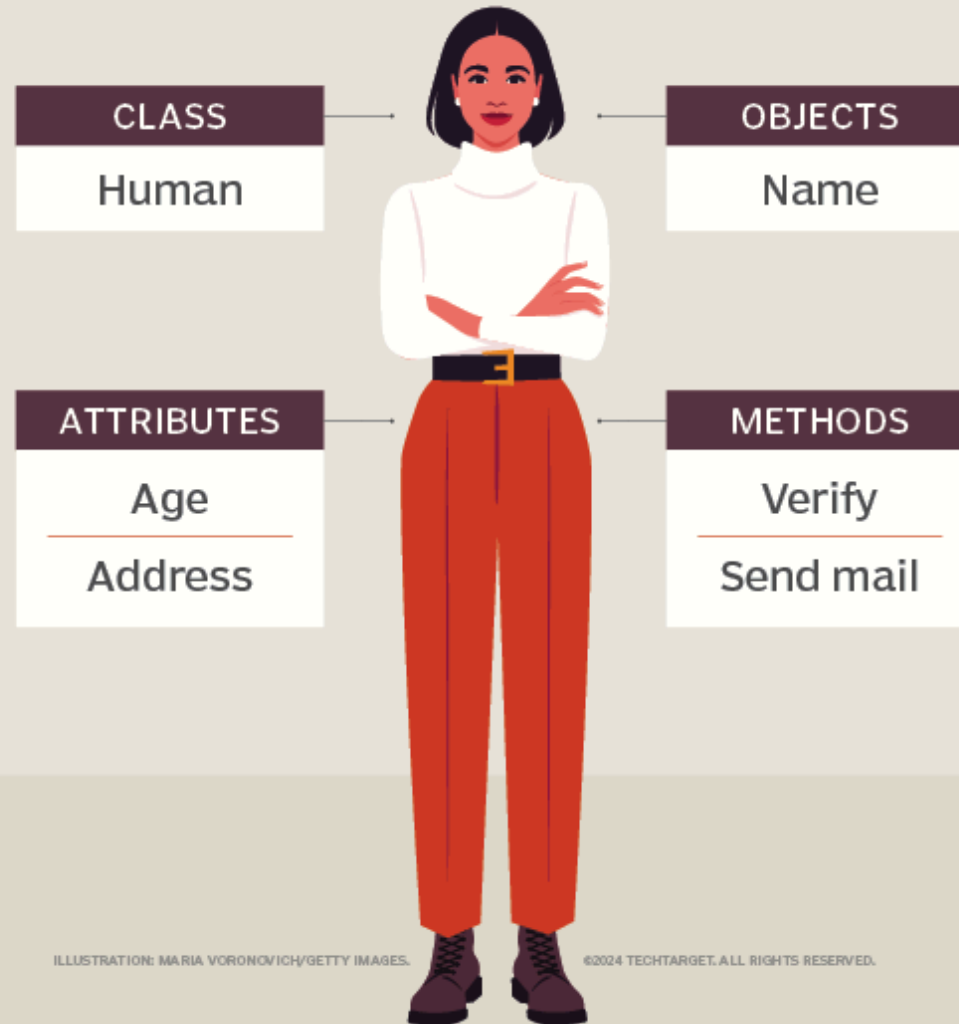
Dealing with corporate that does not understand the concept of standard deviation and how that blurs the "model B is better than model A". And since that statement came up: overfit models.

Can't go on, gotta write a report for corporate, peace 🕊

+ 43 ↓ Reply Award Share ...

Don't take it from me...

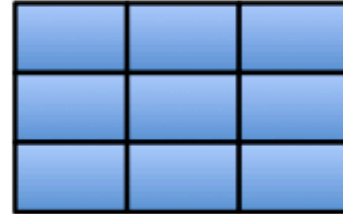
Object-oriented programming



R Objects



Vector

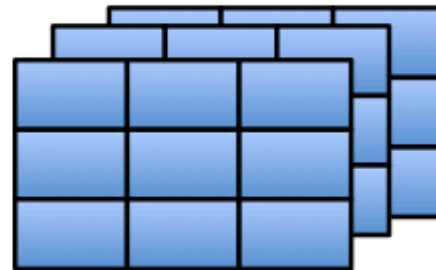


Matrix

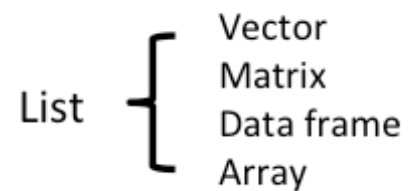


Data frame

*Columns can be different modes



Array



Source: R in Action, p.23

R Objects

Vectors

- one-dimensional
- one mode (e.g., numeric, character, logical)

Matrix

- two-dimensional
- one mode

Array

- multidimensional
- one mode

Lists

- one-dimensional
- one **object** *per element*

Data Frame

- two-dimensional
- one mode *per column*

What's in a name?

Conventions

- assign to an object using ``<-``
- do ***not*** use ``=``

camelCase

PascalCase

snake_case

snake.case

works in R but can get
confusing with Python



Part 2

Steps to Read in Data

1 Find the data

3 Load data

2 Set path to data

1. Find the data

- Downloads folder?
- Working directory?
- Data folder?
- Some other folder?

2. Set path to data

Paths: specify the unique location of a file or directory within your filesystem

- **Path syntax**
 - directory:
 - Windows: use "\\"
 - Macs: use "/"
 - same directory: "./"
 - "up" a directory: "../"
- **Where does this path go: ../../data/my_data.csv**
- **What directory are you in now: ``getwd()``**
- **Temporary solution:** Use ``file.choose()`` to work with a GUI

2. Set path to data

Extension: a suffix at the end of a filename that indicates the file's format or type

- .csv (comma separated)
- .txt
- .xlsx, .xls (Excel)
- .Rdata, .rds (R)
- .sav (SPSS)
- .dat (Mplus, STATA)
- .sas7bdat (SAS)
- .mat (MATLAB)

3. Load the data

- 1 Find the function and sometimes package (use generative AI to help)
- 2 Read the function documentation: `?read.csv``
- 3 Know how the data gets read in
 - Does it get assigned to an object?
 - Will it be a matrix, data frame or list?
 - Once loaded, did it get read in correctly?

On {packages} and `functions()`

Packages

- a collection of pre-defined functions
- denoted with curly brackets: {package}
- **installation:** ``install.packages("package")``
- **usage:** ``library("package")``

Functions

- pre-programmed routine that take some *input*, manipulates it, and produces some *output*
- denoted with a pair of grave accents and parentheses: ``function()``
- **arguments:** pre-defined routines that can be changed based on desired manipulations
- **usage:** ``function(data, argument1 = "do_this", argument2 = "do_that")``

A full-page photograph of a rodeo event. A cowboy in a light blue shirt, jeans, and a straw hat is riding a grey horse, chasing a dark-colored calf. The cowboy is holding a lasso that is looping in the air. The scene is set in a dirt arena with a crowd of spectators in the background under a clear blue sky. Overlaid on the image are text elements: 'Data Wrangling' in a large white box, and 'you' and 'data' in teal boxes pointing to the cowboy and calf respectively. A logo for 'beautiful.ai' is in the bottom right corner.

Data Wrangling

you

data

Data Wrangling

process of transforming raw data into a clean, structured, and usable format for analysis

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280425583

variables

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280425583

observations

country	year	cases	population
Afghanistan	99	745	19987071
Afghanistan	00	2666	20595360
Brazil	99	37737	172006362
Brazil	00	80488	174504898
China	99	212258	1272915272
China	00	213766	1280425583

values

Subsetting

removing missing columns and/or rows

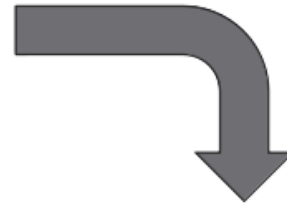
percent_below	percent_approaching
19.7	53.7
24.2	48.9
27.2	55.3
NA	NA
NA	NA
28.6	53.6
NA	NA
NA	NA
5.9	47.1
28.1	49.6
17.1	52.4
NA	NA
NA	NA
19.3	54.1
31.1	52.0
55.7	35.0
38.1	34.8
42.1	32.5



percent_below	percent_approaching
19.7	53.7
24.2	48.9
27.2	55.3
28.6	53.6
5.9	47.1
28.1	49.6
17.1	52.4
19.3	54.1
31.1	52.0
55.7	35.0
38.1	34.8
42.1	32.5

Renaming

percent_below	percent_approaching
19.5	47.7
30.1	50.5
30.4	51.2
30.1	51.7
33.9	50.0
35.8	47.9
39.9	46.1
8.9	42.1
9.3	34.4
31.7	50.3
26.7	51.2
16.1	47.6
14.2	46.8



below	approaching
19.5	47.7
30.1	50.5
30.4	51.2
30.1	51.7
33.9	50.0
35.8	47.9
39.9	46.1
8.9	42.1
9.3	34.4
31.7	50.3
26.7	51.2
16.1	47.6
14.2	46.8

Creating New Variables

separating a character variable



pop_icons	first	last
Ariana Grande	Ariana	Grande
Taylor Swift	Taylor	Swift
Chris Strauss	Chris	Strauss
Olivia Rodrigo	Olivia	Rodrigo
Alex Christensen	Alex	Christensen
Beyonce	Beyonce	NA
Britney Spears	Britney	Spears

Creating New Variables

standardizing a variable

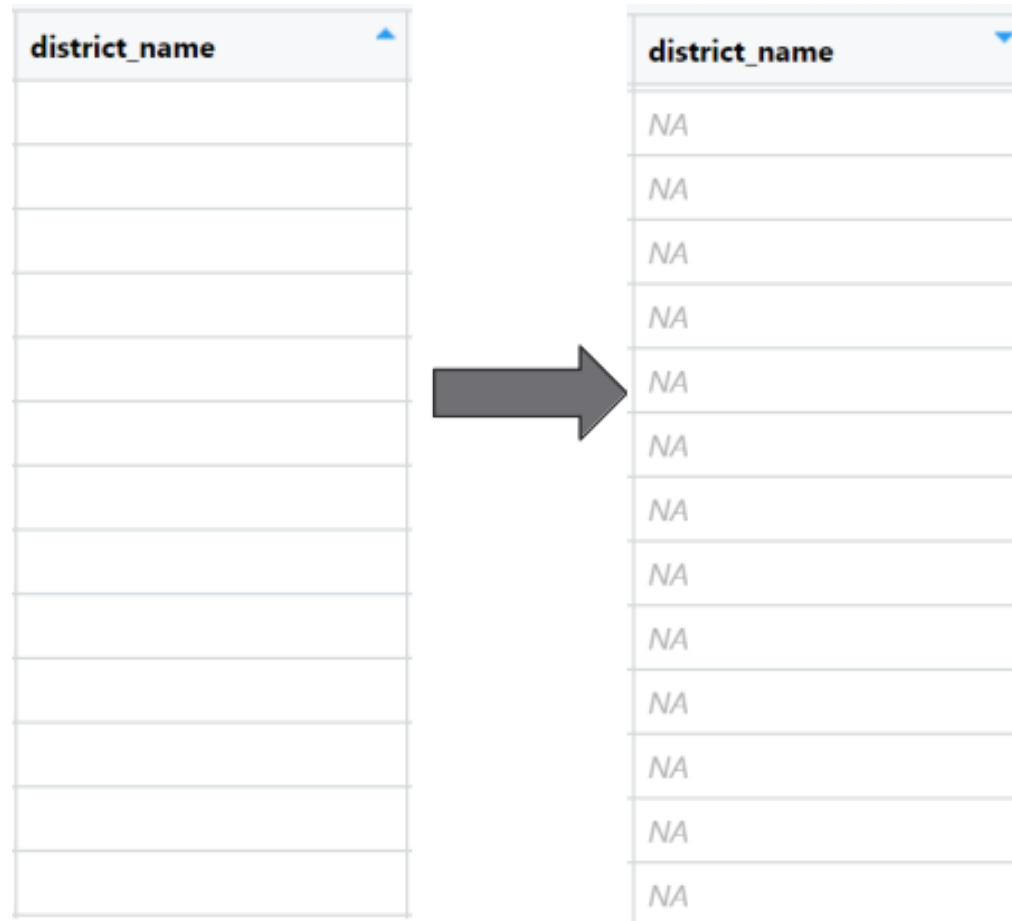


percent_below
19.5
30.1
30.4
30.1
33.9
35.8
39.9
8.9
9.3
31.7
26.7
16.1
14.2

percent_below_z[,1]
-0.36747665
0.42210618
0.44445286
0.42210618
0.70516417
0.84669317
1.15209784
-1.15705948
-1.12726390
0.54128849
0.16884376
-0.62073907
-0.76226806

Adding/Imputing Variables

coding missing values



The diagram illustrates the process of adding or imputing variables by coding missing values. It consists of two tables connected by a large grey arrow pointing from left to right. The left table has a header 'district_name' and 15 empty rows. The right table has the same header 'district_name' and 15 rows, each containing the text 'NA', representing missing values.

district_name

district_name
NA
NA
NA
NA
NA
NA
NA
NA
NA
NA
NA
NA
NA
NA
NA
NA

Adding/Imputing Variables

appending additional observations

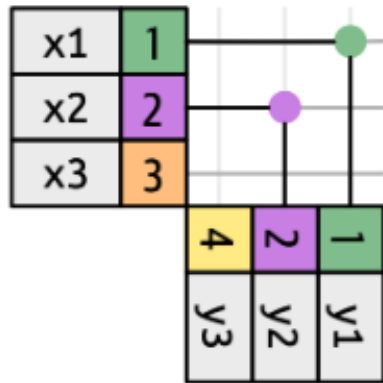


pop_icons
Ariana Grande
Taylor Swift
Chris Strauss
Olivia Rodrigo
Alex Christensen
Beyonce
Britney Spears

pop_icons
Ariana Grande
Taylor Swift
Chris Strauss
Olivia Rodrigo
Alex Christensen
Beyonce
Britney Spears
Andres Gannon

Joining Data Frames

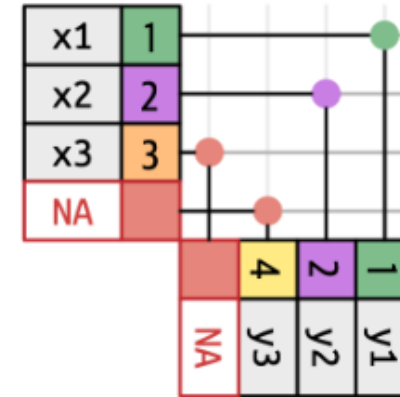
inner_join(x, y)



key	val_x	val_y
1	x1	y1
2	x2	y2

Inner Join

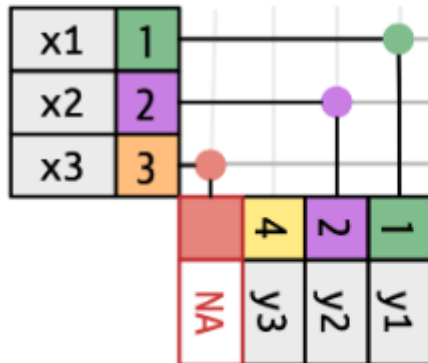
full_join(x, y)



key	val_x	val_y
1	x1	y1
2	x2	y2
3	x3	NA
4	NA	y3

Full Join

left_join(x, y)



key	val_x	val_y
1	x1	y1
2	x2	y2
3	x3	NA

Left Join

right_join(x, y)



key	val_x	val_y
1	x1	y1
2	x2	y2
4	NA	y3

Right Join



Part 2 Cont.